

★ WEEK 15 · LECTURE

The Frontier

Temporal Fusion Transformers, Graph Neural Nets — and AI Agents

Expanded edition · 38 slides · with three hands-on labs

A tour of three cutting-edge ideas, built entirely from tools you already know.
Each one extends a method from earlier weeks — and each comes with honest caveats.

Where We Are Going Today

Nothing here is truly new — it's clever combinations of the building blocks you already own

1

TFT — smarter time-series forecasting

Attention (Week 12) applied to predicting the future, using past AND known-future events.

2

GNN — learning from relationships

Stocks as a network: each one influenced by its neighbors. Extends clustering (Week 9).

3

AI Agents — LLMs that act

An LLM that uses tools in a loop. Combines RAG (Week 12) with RL-style acting (Week 13).

By the end you will forecast with attention weights, spread a shock through a graph, and price an agent's tool call.

1

PART 1 · TFT

Temporal Fusion Transformers

Attention for forecasting

What's Wrong With Simple Forecasting

Most models miss two obvious things

A plain model treats every past day equally and ignores the calendar ahead. But you KNOW some past days mattered more — and you KNOW about future holidays and earnings dates.

What a TFT adds

- Attention: focus on the past days that matter most.
- Known-future inputs: use the calendar you already have.
- Built from the Transformer (Week 12) + nets (Week 8).

The bonus

- It can tell you WHICH inputs drove each forecast.
- Built-in explainability (echoes Week 14's SHAP goal).
- Forecasts come with a sense of certainty, not just a number.

Classic Forecasting vs. TFT

Same question — 'what comes next?' — two very different pairs of eyes

	Classic (moving average, ARIMA-style)	TFT
How it sees the past	fixed recipe — recent days, fixed weights	attention LEARNS which days matter, per forecast
The calendar ahead	invisible	known-future inputs: holidays, earnings dates
Extra inputs	awkward beyond a few	price, volume, macro, static traits — together
Explains itself	the recipe is the explanation	shows the attention weights it used
Appetite	works on short series	data-hungry — the usual deep-learning bill

When data is thin, classic wins on humility. When data is rich, TFT's learned focus earns its keep.

TFT — Focus on What Matters, Past and Future

Attention picks the important past days; known-future events feed in too. Week 12's attention, pointed at forecasting.

TFT — a Forecaster That Knows Which Days Matter

Past days (size = how much the model focuses on each):



A smart predictor for time series

TFT uses ATTENTION (Week 12) to decide which past days matter most — and it also uses KNOWN future events (holidays, scheduled earnings) that a plain model would ignore.

Bonus: it can show WHICH inputs drove the forecast — built-in explainability.

The Idea in One Line

A forecast is a weighted blend of the past

$$\hat{y} = \sum_t (\text{attention}_t) \times (\text{past value}_t)$$

Reading it

- Each past value gets an attention weight.
- Important days get big weights; the rest get small ones.
- The forecast is the weighted sum — just like attention in Week 12.

Why it's strong

- Learns the weights automatically from data.
- Handles many inputs (price, volume, calendar) at once.
- Often beats classic models on rich data — when you have enough of it.

One Attention Forecast, by Hand

past = (10.0, 10.5, 9.8, 12.0, 11.2, 13.5) — three ways to weight it (full version: problem set)

Weighting	The weights a	Forecast $\hat{y} = \sum a \cdot y$	Reading
Flat (no attention)	1/6 each	$67.0 / 6 = 11.17$	every day equal — the plain average
Recency attention	$(1..6)/21$	$245.4 / 21 = 11.69$	recent days pull the forecast up
All on the last day	$(0,...,0,1)$	13.50	attention at its most extreme

A real TFT LEARNS the weight row from data instead of assuming it — that single upgrade is the whole idea.

The Three Input Types a TFT Juggles

Knowing which is which is most of the modeling work

Input type	Examples	Known at forecast time?
Past observed	prices, volume, volatility, past demand	only their history
Known future	holidays, earnings dates, contract resets, seasons	YES — the calendar is certain
Static	sector, country, listing venue, share class	fixed traits, always known

The classic sin is hiding 'known future' inside 'past observed' — the TFT's separation is why it can plan around an earnings date.

2

PART 2 · GNN

Graph Neural Networks

Learning from relationships

Stocks Don't Live Alone

They sit in a web of suppliers, customers, and rivals

Every model so far treated each stock as an island with its own features. But a chip shortage at one supplier ripples to everyone who depends on it. Relationships carry information.

The graph view

- Nodes = stocks (or companies, people, accounts).
- Edges = relationships (supplier, sector, ownership).
- The structure itself is valuable data.

Where it helps

- Supply-chain risk and contagion.
- Fraud rings (accounts linked by money flows).
- Spotting hidden connections between companies.

Finance Is Already Full of Graphs

Four networks your desk touches every day — each with a question only structure can answer

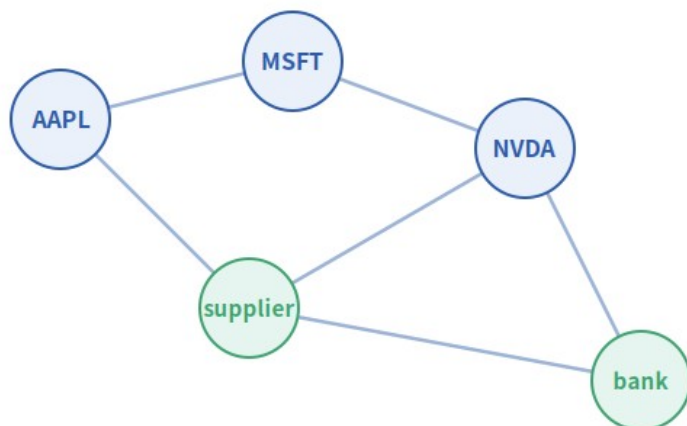
Network	Nodes	Edges	The graph-only question
Supply chain	companies	supplier → customer	who inherits this chip shortage?
Ownership	funds, firms, people	stakes, control	where does risk REALLY concentrate?
Payments	accounts	transfers	is this cluster a fraud ring?
Co-movement	stocks	high correlation	which link carries the contagion?

Tabular models see the nodes' columns; only a graph model can read the wiring between them.

GNN — Each Node Learns From Its Neighbors

Information flows along the edges. 'You are influenced by your connections' — trouble next door becomes a signal for you.

GNN — Learn From a Web of Relationships



"You are influenced by your connections"

Most models treat each stock alone. But stocks live in a web: suppliers, customers, rivals, sectors.

A GNN lets each node (stock) mix in information from its NEIGHBORS, round after round.

So trouble at a supplier can flag risk for the companies that depend on it.

Builds on the clustering idea from Week 9 — but keeps the links.

How a GNN Updates a Node

Blend your own info with your neighbors'

$$h_v^{\text{new}} = \text{combine}\left(h_v, \sum_{u \in \text{neighbors}} h_u\right)$$

Reading it

- h_v is what we know about node v (a stock).
- We gather the info from all its neighbors and combine.
- Repeat over a few rounds so info spreads through the web.

The intuition

- Round 1: you hear from direct neighbors.
- Round 2: you hear from neighbors-of-neighbors.
- It's clustering (Week 9) that keeps the links.

A Shock Spreading, by Hand

Four nodes; A takes a shock of 5. Update: $h \leftarrow \frac{1}{2}h + \frac{1}{2} \cdot (Ah)/\text{deg}$ (full version: problem set)

Round	A (deg 2)	B (deg 2)	C (deg 3)	D (deg 1, linked only to C)
Start	5.00	0	0	0
1	2.50	1.25	0.83	0 — hasn't heard yet
2	1.77	1.46	1.04	0.42 — the shock arrives
3	1.51	1.43	1.13	0.73 — still catching up

D is two hops from A — and the shock reaches it exactly in round 2. Rounds of message-passing = radius of hearing.

3

PART 3 · AI AGENTS

AI Agents

LLMs that use tools and act

From Talking to Doing

A chatbot answers; an agent gets things done

The Week 12 chatbot could only talk. An AI agent can also ACT: search the web, run code, query a database. It works in a loop — think, use a tool, look at the result, decide the next step — until the task is done.

What makes it an agent

- An LLM as the 'brain' that plans.
- A set of TOOLS it can call.
- A loop that lets it react to results and keep going.

It reuses what you know

- RAG (Week 12): look up real facts with a tool.
- RL-style acting (Week 13): choose actions toward a goal.
- The LLM (Week 12) ties it all together.

Chatbot vs. Agent — the Upgrade, Itemized

Same brain, new hands — and therefore new dangers

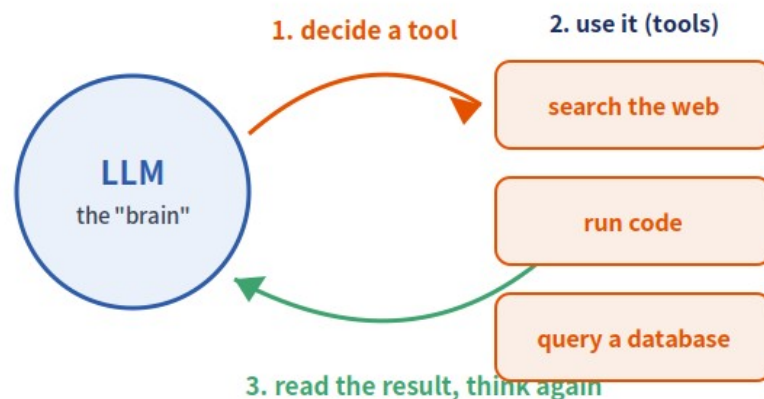
	Week 12 chatbot	Week 15 agent
Can it act?	no — words only	yes — search, code, databases, files
Memory of the task	one exchange at a time	a loop: result feeds the next thought
Failure mode	says something wrong	DOES something wrong
Grounding	RAG on a fixed corpus	RAG plus live tools it chooses itself
Supervision needed	review the answer	gate the ACTIONS — before they run

Every upgrade on the left column doubles as a new obligation in the last row.

The Agent Loop

Think, pick a tool, use it, read the result — then repeat. This loop takes an LLM from answering to completing tasks.

AI Agent — an LLM That Can Use Tools and Act in a Loop



From answering to DOING

A plain LLM just talks. An agent can act:

think -> pick a tool -> use it -> read the result -> decide the next step. Repeat.

It combines RAG (Week 12, look things up) with RL-style acting (Week 13).

Example: "Research a stock" → search, read, compute, then write a report — on its own.

The Loop, Step by Step

A simple cycle that produces complex behavior

think → use tool → observe → repeat

Example: 'Research this stock'

- Think: 'I need recent news and the latest filing.'
- Tools: search the web; read the filing (RAG); run code for ratios.
- Then write a short, sourced summary.

Why guardrails matter

- An agent that can act can also act **WRONGLY**.
- Never let it trade real money unsupervised.
- Limit its tools; log everything; keep a human in charge — capability without control is a liability (Week 14).

Why the Agent Should Look It Up — by Hand

Task: state last quarter's EPS. Being right is worth 10; being wrong costs 20; each tool call costs 1 (full version: problem set)

Strategy	P(correct)	$EV = P \cdot 10 - (1-P) \cdot 20 - \text{cost}$	Verdict
Answer from memory	0.60	$6 - 8 - 0 = -2.0$	negative — hallucination risk dominates
RAG: read the filing	0.95	$9.5 - 1 - 1 = +7.5$	one tool call flips the sign
Search + RAG	0.99	$9.9 - 0.2 - 2 = +7.7$	the extra call still pays — barely

Drop the stakes to $L = 2$ and memory turns positive — the right tooling depth depends on what a wrong answer costs.

The Guardrail Kit

Five risks an acting agent creates — and the standing rule for each

Risk	What it looks like	The standing rule
Unsupervised trading	'it seemed confident, so it executed'	agents propose; humans approve — always
Tool sprawl	one prompt away from your whole database	allowlist tools per task; least privilege
Silent actions	nobody can reconstruct what it did	log every tool call, input, and output
Runaway loops	tool fails → retries forever, bills climbing	step limits and spend caps, hard-coded
Confident nonsense	fluent summary of a misread filing	require citations; spot-check (Week 12's valve)

The agent inherits every LLM weakness from Week 12 — and adds consequences to them.

4

PART 4 · STAYING GROUNDED

Staying Grounded

Promise, caveats, and your turn

Exciting Tools, Same Old Rules

The frontier is powerful, but the fundamentals still govern it. Fancy models don't repeal Week 1 or Week 10.

Exciting — but Stay Grounded

The promise

TFT: better, explainable time-series forecasts.

GNN: capture supply chains and contagion.

Agents: automate whole research workflows.

Genuinely powerful new capabilities.

The honest caveats

Data-hungry — finance often has too little.

Complex to build, tune, and maintain.

Agents can act wrongly — need guardrails.

Still bound by Week 1 and Week 10's rules.

Every Frontier Claim, and the Week That Governs It

New models, old laws — the course's rules apply verbatim

The frontier claim	The governing law	From
'TFT beat ARIMA in our backtest'	purge the CV, deflate for luck, count the trials	W10
'The GNN found hidden contagion links'	correlation edges are rung 1 — confound-check them	W14
'The agent automated the research desk'	log, gate, and audit — explanation before deployment	W14
'It needs more data than we have'	data hunger is a cost, not a footnote	W8
'The demo looked amazing'	process beats genius; demos are marketing	W1

If a frontier pitch can't survive this table, it isn't a frontier — it's a brochure.

The Frontier in Our Three Apps

Where each new tool fits best

Trading & forecasting

TFT + agents

TFT forecasts with calendar-aware inputs; GNNs model sector and supply-chain contagion; agents automate research with a human approving trades.

forecast → context → workflow

Fraud detection

the GNN home game

Catch fraud RINGS via linked accounts — relationships reveal what single accounts hide. Agents investigate an alert by pulling records.

structure is the signal

Chatbot & research

agents

Turn the chatbot into a research assistant: search, read filings (RAG), compute, summarize — always sourced, always reviewed.

talking → doing, gated

Same three apps as all course — now supercharged, but bound by the same discipline.

Everything Connects Back

The frontier is just your toolkit, recombined

1

TFT = attention + neural nets

Week 12's attention and Week 8's networks, aimed at forecasting with calendar-aware inputs.

2

GNN = clustering that keeps the links

Week 9's grouping idea, but information flows along real relationships instead of being discarded.

3

Agent = LLM + RAG + RL-style acting

Week 12's LLM and retrieval, plus Week 13's act-toward-a-goal loop, wired into a tool-using system.

Master the fundamentals and the frontier becomes readable — not magic, just new combinations.

Fifteen Weeks, One Machine

Every part has a station — and next week you run the whole line

Station	Weeks	What it contributes
Foundations	W1–W2	process beats genius; data is biased until proven otherwise
Labels & models	W3–W8	honest targets, then learners: linear → trees → nets
Structure	W9, W11	clusters and portfolios — order without labels
Honesty	W10	purged CV, deflated Sharpe — the license to believe
Language & action	W12–W13	LLMs, RAG, RL — reading and acting
Trust	W14	cause, explanation, fairness — the license to deploy
Frontier	W15	the same parts, recombined: TFT, GNN, agents

Final project next week: one end-to-end pipeline through every station above.

Which Frontier Tool? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Tool / diagnosis
Forecast demand around known holiday spikes	TFT — that's a known-future input, its home turf
Five shell accounts moving money in a circle	GNN — the ring is invisible row-by-row, obvious as a graph
Summarize 40 filings with sourced citations	Agent — search + RAG + code, human reviewing
'The agent placed the trade itself'	Guardrail breach — propose/approve was the standing rule
'Our TFT crushed the backtest' (30 tickers, 2 years)	W10 + W8 — data-hungry model, luck-prone test

If you got 4 of 5, the frontier-reader's instinct of Week 15 is yours.

Getting This Week's Data

Tiny inline demos — no heavy libraries needed

Built-in toy examples

- Real TFT/GNN code needs big specialist libraries.
- So we capture the CORE IDEA with a few lines of numpy.
- Lab 1: attention forecast · Lab 2: a tiny graph update.

What to take away

- You won't build a production TFT today — you'll SEE the mechanism.
- That intuition is what lets you use the real tools later.
- The ideas scale up; the principles stay the same.

[CODE] `colab_setup.py` · tiny inline demos (no download)

```
import numpy as np
rng = np.random.default_rng(0)
# Lab 1: a short series of past values to forecast from
past = np.array([10.0, 10.5, 9.8, 12.0, 11.2, 13.5]) # last day matters most
```

Hands-On Lab 1 — Attention-Style Forecast

Weight the past, focus on what matters

[CODE] colab_lab1.py · tiny inline demo (no download)

```
import numpy as np
past = np.array([10.0, 10.5, 9.8, 12.0, 11.2, 13.5])

# attention: more weight on recent days (TFT-style idea)
raw = np.array([1, 2, 3, 4, 5, 6.0]) # recency score
attn = raw / raw.sum()             # weights sum to 1

forecast = np.sum(attn * past)      # weighted blend
print('attention weights:', attn.round(2))
print('forecast:', round(forecast, 2))
# try flat weights (no attention) and compare
```

Your task (step by step)

- Give each past day an attention weight.
- Make recent days matter more.
- Forecast = weighted blend of the past.
- Change the weights and watch it shift.
- Q: how do the weights change the forecast?

Hands-On Lab 2 — A Tiny Graph Update

Each node mixes in its neighbors' values

[CODE] colab_lab2.py · tiny inline demo (no download)

```
import numpy as np
# 4 nodes; A[i,j]=1 means i and j are linked (a GNN step)
A = np.array([[0,1,1,0],
              [1,0,1,0],
              [1,1,0,1],
              [0,0,1,0]], float)
h = np.array([5.0, 0.0, 0.0, 0.0]) # node 0 gets a shock

for step in range(3):
    # spread over 3 rounds
    h = 0.5*h + 0.5*(A @ h)/A.sum(1) # blend w/ neighbors
    print('round', step+1, h.round(2))
```

Your task (step by step)

- Define a small network (who links to whom).
- Give each node a starting value.
- Update each node by averaging its neighbors.
- Watch a shock spread across the graph.
- Q: how far does one node's change travel?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 and Lab 2 code above, one plain-English sentence per line.
- 2) Verify my hand-calcs: (a) flat 11.17 / recency 11.69 / last-only 13.50 on the past series; (b) the 4-node shock: node D = 0 / 0.42 / 0.73 after rounds 1-3.
- 3) Learned attention: fit weights on a toy series where day t-1 and day t-5 truly matter (make one); show the learned weights find both days.
- 4) Graph: add an edge B-D and rerun; show the shock now reaches D in round 1 - structure is the signal.
- 5) Agent EV: recompute my tool-choice table for L in [2, 20, 100] and print which strategy wins at each stake.
- 6) In 3 sentences: why all three are recombinations of earlier weeks, naming the weeks.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 What earlier idea does a TFT mainly build on?

- A** K-Means clustering
- B** Attention from Transformers (Week 12) ✓
- C** The Sharpe ratio
- D** Purged CV

Answer: A TFT uses attention — the Week 12 Transformer idea — to focus on the past days that matter for forecasting.

Q2 What makes an AI agent different from a plain chatbot?

- A** It is smaller
- B** It can use tools and act in a loop ✓
- C** It never makes mistakes
- D** It needs no LLM

Answer: An agent uses tools (search, code, databases) in a think-act-observe loop, going from talking to doing.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 In the 4-node graph, node D first moved in round 2. Why exactly then?

- A** Its degree is 1
- B** D is two hops from the shock — one hop of news per round ✓
- C** The 0.5 blend is too slow
- D** Rounding error

Answer: Each round passes messages one edge. D touches only C, and C heard in round 1 — so D hears in round 2.

Q4 EPS question, wrong answers cost 20. Memory: $P = 0.6$, free. The agent should:

- A** Answer from memory — it's free
- B** Call the RAG tool: EV -2.0 vs $+7.5$ ✓
- C** Refuse the task
- D** Raise the temperature

Answer: Memory's EV is negative at these stakes; one tool call flips the sign. Tooling depth follows the cost of being wrong.

Week 15 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
TFT	attention (W12) aimed at time: learned weights over the past + known-future inputs.
Its appetite	data-hungry and W10-governed — thin data keeps classic models honest.
GNN	$h \leftarrow \frac{1}{2}h + \frac{1}{2} \cdot (Ah)/\text{deg}$: each round, nodes hear one hop farther. W9 with links kept.
Its home games	supply chains, contagion, fraud rings — questions ABOUT the wiring.
Agent	LLM brain + tools + think-act-observe loop = RAG (W12) + acting (W13).
Its leash	propose/approve, tool allowlists, logs, step caps — capability needs control (W14).

Next stop: the Final Project — one end-to-end pipeline through everything you've built.

Three Frontiers, One Picture Each

The frontier is recombination, not magic. Your toolkit already contains the pieces

1

TFT = attention for forecasting

Focus on the past days that matter and use known-future events. Explainable, strong on rich data.

2

GNN = learning from the web of links

Each node mixes in its neighbors. Captures supply chains, contagion, and fraud rings.

3

AI Agent = an LLM that acts

Think, use a tool, observe, repeat. Combines RAG and RL-style acting — powerful, but needs guardrails.

Keep the pictures: weighted eyes on the past, a shock crossing a web — and a brain with gated hands.

Week 15 — Three Things to Remember

Before next class: finish Labs 1–3 and start thinking about your final-project idea

1

TFT brings attention to time series

Weight the important past days, use the known calendar ahead, and explain the forecast. Data-hungry.

2

GNN learns from relationships

Stocks as a network; each node absorbs its neighbors' information. Great for contagion and connection.

3

Agents turn LLMs into doers

Tools + a think-act loop = automated workflows. Keep tight guardrails and a human in charge.

Next week — Final Project: bring the whole course together into one end-to-end pipeline.